Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic

Code:

```
#include<stdio.h>
int gcd(int a,int b){return b==0?a:gcd(b,a%b);}
int lcm(int a,int b){return (a*b)/gcd(a,b);}
int main(){
int n,i,j,time=0;
printf("Enter the number of processes:");
scanf("%d",&n);
int burst[n],period[n],rem[n];
printf("Enter the CPU burst times:\n");
for(i=0;i<n;i++)scanf("%d",&burst[i]);
printf("Enter the time periods:\n");
for(i=0;i<n;i++)scanf("%d",&period[i]);
for(i=0;i<n;i++)rem[i]=0;
int hyper=period[0];
for(i=1;i<n;i++)hyper=lcm(hyper,period[i]);
printf("LCM=%d\n\n",hyper);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for(i=0;i<n;i++)printf("%d\t%d\t%d\n",i+1,burst[i],period[i]);
printf("\nScheduling occurs for %d ms\n\n",hyper);
while(time<hyper){
for(i=0;i<n;i++){
if(time%period[i]==0)rem[i]=burst[i];
}
int idx=-1,minp=9999;
for(i=0;i<n;i++){
if(rem[i]>0 && period[i]<minp){
minp=period[i];
idx=i;
}
```

```
}
if(idx==-1){
printf("%dms onwards: CPU is idle\n",time);
}
else{
printf("%dms onwards: Process %d running\n",time,idx+1);
rem[idx]--;
}
time++;
}
return 0;
}
```

Output:

```
C:\Users\BMSCECSE-I4\Deskt

Enter the number of processes:2
Enter the CPU burst times:
25
40
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       25      50
2       40      100

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
14ms onwards: Process 1 running
15ms onwards: Process 1 running
16ms onwards: Process 1 running
17ms onwards: Process 1 running
18ms onwards: Process 1 running
19ms onwards: Process 1 running
20ms onwards: Process 1 running
21ms onwards: Process 1 running
22ms onwards: Process 1 running
23ms onwards: Process 1 running
24ms onwards: Process 1 running
25ms onwards: Process 2 running
26ms onwards: Process 2 running
27ms onwards: Process 2 running
28ms onwards: Process 2 running
29ms onwards: Process 2 running
30ms onwards: Process 2 running
31ms onwards: Process 2 running
32ms onwards: Process 2 running
33ms onwards: Process 2 running
34ms onwards: Process 2 running
```

b) Earliest-deadline First

Code:

```
#include <stdio.h>
#define MAX 10
int main() {
printf("Naysha Navin Shrivastava 1WA24CS184 \n");
int n;
int burst[MAX], deadline[MAX], period[MAX];
int remaining[MAX];
int time = 0;
printf("Enter number of tasks: ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
printf("Task %d (burst deadline period): ", i + 1);
scanf("%d %d %d", &burst[i], &deadline[i],
&period[i]); remaining[i] = 0; // initially no job released
}
printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
}
int limit = 20;
printf("\nScheduling occurs for %d ms\n\n", limit);
while (time < limit) {
for (int i = 0; i < n; i++) {
if (time % period[i] == 0) {
remaining[i] = burst[i];
}
}
int min_deadline = 9999;
int selected = -1;
for (int i = 0; i < n; i++) {
if (remaining[i] > 0) {
```

```
if (deadline[i] < min_deadline) {
min_deadline = deadline[i];
selected = i;
}
}
}
if (selected == -1) {
printf("%dms : CPU is idle.\n", time);
} else {
printf("%dms : Task %d is running.\n", time, selected + 1);
remaining[selected]--;
}
time++;
}
return 0;
}
```

Output:

```
C:\Users\BMSCECSE-l4\Deskt

Naysha Navin Shrivastava 1WA24CS184
Enter number of tasks: 2
Task 1 (burst deadline period): 2
4

5
Task 2 (burst deadline period): 3
7
20

PID     Burst   Deadline        Period
1       2       4               5
2       3       7               20

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : CPU is idle.
8ms : CPU is idle.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : CPU is idle.
13ms : CPU is idle.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.

Process returned 0 (0x0)   execution time : 37.760 s
Press any key to continue.
```